PPL Week-5

Rachit Takate – 230962294 – B54

Ia. Write a program in CUDA to add two vectors of length N using block size as N.

```
#include <stdio.h>
#include <stdlib.h>
#include <cuda.h>

__global__ void add(int *a, int *b, int *c) {
    int tid = blockDim.x * blockIdx.x + threadIdx.x;

    c[tid] = a[tid] + b[tid];
}

int main() {
    int *a, *b, *c;
    int *dA, *dB, *dC;
    int n;

    printf("Array size: ");
    scanf("%d", &n);
    size_t size = sizeof(int) * n;

    a = (int *) malloc(size);
    b = (int *) malloc(size);
    c = (int *) malloc(size);
    for(int i = 0; i < n; i++) {
        a[i] = i + 1;
        b[i] = 3 - 2 * i;
    }

    cudaMalloc((void **) &dA, size);
    cudaMalloc((void **) &dB, size);
    cudaMalloc((void **) &dC, size);

    cudaMemcpy(dA, a, size, cudaMemcpyHostToDevice);
    cudaMemcpy(dB, b, size, cudaMemcpyHostToDevice);

    add<<<1, n>>>(dA, dB, dC);
    cudaMemcpy(c, dC, size, cudaMemcpyDeviceToHost);

    printf("Result:\n");
    for(int i = 0; i < n; i++)
        printf("%d ", c[i]);
    printf("\n");

    cudaFree(dA);
    cudaFree(dB);
    cudaFree(dC);

    printf("Rachit 54\n");
    return 0;
}
```

Output

```
(base) mca@computinglab26-26:~/230962294/PP Lab/Week5$ nvcc Q1a.cu
(base) mca@computinglab26-26:~/230962294/PP Lab/Week5$ ./a.out
Array size: 64
Result:
4 3 2 1 0 -1 -2 -3 -4 -5 -6 -7 -8 -9 -10 -11 -12 -13 -14 -15 -16 -17 -18 -19 -20 -21 -22 -23 -24 -25 -26 -27 -28 -29 -30 -31 -32 -33 -34 -35 -36 -37 -38 -39 -40 -41 -42 -43 -44 -45 -46 -47 -48 -49 -50 -51 -5
2 -53 -54 -55 -56 -57 -58 -59
Rachit 54
```

Ib. Write a program in CUDA to add two vectors of length N using N threads.

```
#include <stdio.h>
#include <stdlib.h>
#include <cuda.h>

__global__ void add(int *a, int *b, int *c) {
    int tid = blockDim.x * blockIdx.x + threadIdx.x;

    c[tid] = a[tid] + b[tid];
}

int main() {
    int *a, *b, *c;
    int *dA, *dB, *dC;
    int n;

    printf("Array size: ");
    scanf("%d", &n);
    size_t size = sizeof(int) * n;

    a = (int *) malloc(size);
    b = (int *) malloc(size);
    c = (int *) malloc(size);
    for(int i = 0; i < n; i++) {
        a[i] = i + 1;
        b[i] = 3 - 2 * i;
    }

    cudaMalloc((void **) &dA, size);
    cudaMalloc((void **) &dB, size);
    cudaMalloc((void **) &dC, size);

    cudaMemcpy(dA, a, size, cudaMemcpyHostToDevice);
    cudaMemcpy(dB, b, size, cudaMemcpyHostToDevice);

    add<<<n, 1>>>(dA, dB, dC);
    cudaMemcpy(c, dC, size, cudaMemcpyDeviceToHost);

    printf("Result:\n");
    for(int i = 0; i < n; i++)
        printf("%d ", c[i]);
    printf("\n");

    cudaFree(dA);
    cudaFree(dB);
    cudaFree(dC);

    printf("Rachit 54\n");
    return 0;
}
```

Output

```
(base) mca@computinglab26-26:~/230962294/PP Lab/Week5$ nvcc Q1b.cu
(base) mca@computinglab26-26:~/230962294/PP Lab/Week5$ ./a.out
Array size: 64
Result:
4 3 2 1 0 -1 -2 -3 -4 -5 -6 -7 -8 -9 -10 -11 -12 -13 -14 -15 -16 -17 -18 -19 -20 -21 -22 -23 -24 -25 -26 -27 -28 -29 -30 -31 -32 -33 -34 -35 -36 -37 -38 -39 -40 -41 -42 -43 -44 -45 -46 -47 -48 -49 -50 -51 -5
2 -53 -54 -55 -56 -57 -58 -59
Rachit 54
```

II. Implement a CUDA program to add two vectors of length N by keeping the number of threads per block as 256 (constant) and vary the number of blocks to handle N elements.

```
#include <stdio.h>
```

```
#include <cuda.h>

__global__ void add(int *a, int *b, int *c, int n) {
    int tid = blockDim.x * blockIdx.x + threadIdx.x;

    if(tid < n)
        c[tid] = a[tid] + b[tid];
}

int main() {
    int *a, *b, *c;
    int *dA, *dB, *dC;
    int n;

    printf("Array size: ");
    scanf("%d", &n);
    size_t size = sizeof(int) * n;

    a = (int *) malloc(size);
    b = (int *) malloc(size);
    c = (int *) malloc(size);
    for(int i = 0; i < n; i++) {
        a[i] = i + 1;
        b[i] = 3 - 2 * i;
    }

    cudaMalloc((void **) &dA, size);
    cudaMalloc((void **) &dB, size);
    cudaMalloc((void **) &dC, size);

    cudaMemcpy(dA, a, size, cudaMemcpyHostToDevice);
    cudaMemcpy(dB, b, size, cudaMemcpyHostToDevice);

    add<<<(n - 1) / 256 + 1, 256>>>(dA, dB, dC, n);
    cudaMemcpy(c, dC, size, cudaMemcpyDeviceToHost);

    printf("Result:\n");
    for(int i = 0; i < n; i++)
        printf("%d ", c[i]);
    printf("\n");

    cudaFree(dA);
    cudaFree(dB);
    cudaFree(dC);

    printf("Rachit 54\n");
    return 0;
}
```

Output

```
(base) mca@computinglab26-26:~/230962294/PP Lab/Week5$ nvcc Q2.cu
(base) mca@computinglab26-26:~/230962294/PP Lab/Week5$ ./a.out
Array size: 260
Result:
4 3 2 1 0 -1 -2 -3 -4 -5 -6 -7 -8 -9 -10 -11 -12 -13 -14 -15 -16 -17 -18 -19 -20 -21 -22 -23 -24 -25 -26 -27 -28 -29 -30 -31 -32 -33 -34 -35 -36 -37 -38 -39 -40 -41 -42 -43 -44 -45 -46 -47 -48 -49 -50 -51 -5
2 -53 -54 -55 -56 -57 -58 -59 -60 -61 -62 -63 -64 -65 -66 -67 -68 -69 -70 -71 -72 -73 -74 -75 -76 -77 -78 -79 -80 -81 -82 -83 -84 -85 -86 -87 -88 -89 -90 -91 -92 -93 -94 -95 -96 -97 -98 -99 -100 -101 -102 -1
03 -104 -105 -106 -107 -108 -109 -110 -111 -112 -113 -114 -115 -116 -117 -118 -119 -120 -121 -122 -123 -124 -125 -126 -127 -128 -129 -130 -131 -132 -133 -134 -135 -136 -137 -138 -139 -140 -141 -142 -143 -144
 -145 -146 -147 -148 -149 -150 -151 -152 -153 -154 -155 -156 -157 -158 -159 -160 -161 -162 -163 -164 -165 -166 -167 -168 -169 -170 -171 -172 -173 -174 -175 -176 -177 -178 -179 -180 -181 -182 -183 -184 -185 -
186 -187 -188 -189 -190 -191 -192 -193 -194 -195 -196 -197 -198 -199 -200 -201 -202 -203 -204 -205 -206 -207 -208 -209 -210 -211 -212 -213 -214 -215 -216 -217 -218 -219 -220 -221 -222 -223 -224 -225 -226 -22
7 -228 -229 -230 -231 -232 -233 -234 -235 -236 -237 -238 -239 -240 -241 -242 -243 -244 -245 -246 -247 -248 -249 -250 -251 -252 -253 -254 -255
Rachit 54
```

III. Write a program in CUDA to process a 1D array containing angles in radians to generate sine of the angles in the output array. Use appropriate function.

```
#include <stdio.h>
#include <cuda.h>
```

```
#define N 50

__global__ void mySin(float *a, float *b) {
    int tid = blockDim.x * blockIdx.x + threadIdx.x;

    b[tid] = sinf(a[tid]);
}

int main() {
    float a[N], b[N];
    float *dA, *dB;
    size_t size = sizeof(float) * N;
    for(int i = 0; i < N; i++)
        a[i] = i;

    cudaMalloc((void **) &dA, size);
    cudaMalloc((void **) &dB, size);

    cudaMemcpy(dA, a, size, cudaMemcpyHostToDevice);

    mySin<<<(N - 1) / 256 + 1, 256>>>(dA, dB);
    cudaMemcpy(b, dB, size, cudaMemcpyDeviceToHost);

    printf("Result:\n");
    for(int i = 0; i < N; i++)
        printf("%f ", b[i]);
    printf("\n");

    cudaFree(dA);
    cudaFree(dB);

    printf("Rachit 54\n");
    return 0;
}
```

Output

```
(base) mca@computinglab26-26:~/230962294/PP Lab/Week5$ nvcc Q3.cu
(base) mca@computinglab26-26:~/230962294/PP Lab/Week5$ ./a.out
Result:
0.000000 0.841471 0.909297 0.141120 -0.756802 -0.958924 -0.279415 0.656987 0.989358 0.412118 -0.544021 -0.999990 -0.536573 0.420167 0.990607 0.650288 -0.287903 -0.961397 -0.750987 0.149877 0.912945 0.836656
-0.008851 -0.846220 -0.905578 -0.132352 0.762558 0.956376 0.270906 -0.663634 -0.988032 -0.404038 0.551427 0.999912 0.529083 -0.428183 -0.991779 -0.643538 0.296369 0.963795 0.745113 -0.158623 -0.916522 -0.831
775 0.017702 0.850904 0.901788 0.123573 -0.768255 -0.953753
Rachit 54
```